

GUIA DE ESTUDIO · 2026

Java

Apuntes

Completos

De los fundamentos a concurrencia, Streams y buenas prácticas

- ▶ Sintaxis, tipos de datos y control de flujo
- ▶ Programación orientada a objetos (herencia, polimorfismo, interfaces)
- ▶ Excepciones, colecciones, genéricos, lambdas y Streams
- ▶ Concurrencia, hilos, virtual threads y CompletableFuture
- ▶ E/S de ficheros, JDBC, Maven, JUnit 5 y patrones de diseño
- ▶ Chuleta final: sintaxis, APIs clave y errores frecuentes

Basado en Java 21 LTS / 25 LTS · Contenido con ejemplos de código listos para ejecutar

01 Introducción: plataforma Java (JVM, JRE, JDK) →

02 Instalación, primer programa y compilation path →

03 Sintaxis básica, comentarios y convenciones →

04 Variables, tipos de datos y operadores →

05 Control de flujo →

06 Arrays y Strings →

07 Orientación a objetos: clases y objetos →

08 Herencia, polimorfismo, abstracción e interfaces →

09 Enums, records y principios SOLID →

10 Manejo de excepciones →

11 Colecciones (Collection Framework) →

12 Genéricos →

13 Lambdas, funciones funcionales y Streams →

14 Entrada/Salida y ficheros (java.io / java.nio) →

15 Concurrencia y multihilo →

16 JDBC: bases de datos desde Java →

17 Maven, estructura de proyecto y JUnit 5 →

18 Patrones de diseño habituales →

19 Buenas prácticas y errores frecuentes →

20 Chuleta rápida (cheat sheet) →

1 Introducción: la plataforma Java

Java es un lenguaje de programación orientado a objetos, fuertemente tipado, creado por *James Gosling* en Sun Microsystems (1995) y hoy mantenido por Oracle/OpenJDK. Su lema original: "*Write Once, Run Anywhere*" (WORA): se compila una vez a bytecode y se ejecuta en cualquier sistema con JVM.

1.1 Componentes de la plataforma

Componente	Qué es
JDK	Java Development Kit: incluye compilador (<code>javac</code> , ver 11+, <code>java</code> compile-run), depurador, herramientas (<code>jcmd</code> , <code>jlink</code> , <code>jshell</code>) y bibliotecas. <i>Lo necesitas para programar.</i>
JRE	Java Runtime Environment: JVM + bibliotecas estándar. Suficiente para <i>ejecutar</i> , no para compilar.
JVM	Java Virtual Machine: interpreta/JIT-compila el bytecode a código máquina nativo. Gestiona memoria (heap, stack) y el GC.
Bytecode	Salida de <code>javac</code> : ficheros <code>.class</code> con instrucciones para la JVM (p. ej. <code>iadd</code> , <code>invokevirtual</code>).
JAR	Java ARchive: ZIP con clases + manifiesto <code>META-INF/MANIFEST.MF</code> (clase principal).

Cadena de ejecución

fuelle `.java` → `javac` → `.class` (bytecode) → JVM → código máquina nativo. Por eso el mismo `.class` corre en Windows, Linux o macOS.

1.2 Características clave

- **Orientado a objetos** (clases, herencia, encapsulado, polimorfismo).
- **Tipado estático y fuerte**: los tipos se compran en compilación; no hay conversión implícita entre tipos incompatibles.
- **Memoria gestionada**: el programador no libera memoria; el *Garbage Collector* reclama objetos sin referencias.
- **Seguridad**: sin punteros aritméticos, verificación de bytecode, sandbox (SecurityManager, hoy en desuso).
- **Multihilo nativo** + *virtual threads* (JEP 444, estable desde Java 21).
- **Robustez**: recolección de basura, checked exceptions, gestión de memoria por regiones.
- **Portable**: mismos tipos (siempre 32 bits en `int` , etc.) y API estándar en todas las JVM.

1.3 Versiones y LTS

Versión	Año	LTS	Novedades destacadas
8	2014	✓	Lambdas, Streams, <code>Optional</code> , fecha/time (java.time)
11	2018	✓	<code>var</code> en locals, HttpClient, <code>String.isBlank()</code> , LTS a 6 meses
17	2021	✓	<code>sealed</code> , <code>records</code> , <code>pattern matching instanceof</code> , text blocks
21	2023	✓	Virtual threads , <code>switch</code> con patrones, <code>record patterns</code>
25	2025	✓	Compact Object Headers , <code>Scoped Values</code> (preview), mejoras en ZGC

Cual
En proyectos nuevos usa **Java 21 LTS** como mínimo (hoy ya se puede usar 25 LTS). Evita versiones no-LTS en producción.

2 Instalación y primer programa

2.1 Configuración

1. Instala un JDK (Oracle, Temurin/Adoptium, Microsoft OpenJDK, Corretto).
2. Variables de entorno:

```
JAVA_HOME = C:\Program Files\Java\jdk-21
PATH      = %JAVA_HOME%\bin // añadir al PATH del sistema
```

3. Comprueba: `java -version`, `javac -version`.
4. REPL para probar ideas: `jshell` (sale con `/exit`).

2.2 Hola mundo

Archivo `HolaMundo.java` (el nombre **debe coincidir** con la clase pública):

```
public class HolaMundo {
    public static void main(String[] args) {
        System.out.println("Hola, mundo!");
    }
}
```

```
javac HolaMundo.java // produce HolaMundo.class
java  HolaMundo      // ejecuta (sin .class, sin classpath)
java  -cp out HolaMundo // si las clases están en la carpeta "out"
```

Java 11+: un solo comando

`java HolaMundo.java` compila y ejecuta en memoria (source-file mode). Ideal para scripts y pruebas rápidas.

2.3 Firmas válidas de main

```
public static void main(String[] args) // clásica
public static void main(String... args) // varargs (equivalente)
public static void main(String[] args) throws Exception
```

2.4 Paquetes y ejecución con classpath

```
package com.ejemplo.utilidades; // primera línea del fichero

// Estructura de carpetas: com/ejemplo/utilidades/Clase.java
javac -d out com/ejemplo/utilidades/Clase.java
java  -cp out com.ejemplo.utilidades.Clase
```

Reglas del classpath

`-cp` / `-classpath` define dónde buscan JVM y compilador clases y JARs. Separa elementos con `;` en Windows y `:` en Linux/macOS. El separador de paquetes es siempre el punto `.`.

2.5 Estructura de un proyecto (convención Maven)

```
mi-proyecto/  
├─ pom.xml  
└─ src/  
    ├─ main/  
    │   ├─ java/com/empresa/app/Main.java  
    │   └─ resources/    (log4j2.xml, application.properties...)  
    └─ test/  
        ├─ java/com/empresa/app/MainTest.java  
        └─ resources/
```

3 Sintaxis básica y convenciones

3.1 Estructura de un fichero

```
// 1. Declaración de paquete (opcional, solo una)
package com.empresa.modulo;

// 2. Imports (o import static)
import java.util.List;
import static java.lang.Math.PI;

// 3. Una única clase pública (puede haber varias clases no públicas)
public class Ejemplo { /* ... */ }

class Auxiliar { /* clase auxiliar en el mismo fichero */ }
```

3.2 Tipos de comentarios

```
// Comentario de una línea
/* Comentario
   de varias líneas */
/**
 * Javadoc: documentación oficial.
 * @param args argumentos de línea de comandos
 * @return nada
 * @throws Exception si algo falla
 */
```

3.3 Anotaciones (metadatos)

Anotación	Uso
<code>@Override</code>	Indica que sobrescribe un método padre. Usa siempre: el compilador avisa si firmas mal.
<code>@Deprecated</code>	Método/clase obsoleta; el compilador avisa al usarla.
<code>@SuppressWarnings("unused")</code>	Silencia avisos concretos del compilador.
<code>@FunctionalInterface</code>	Marca interfaz con un único método abstracto (para lambdas).
<code>@SafeVarargs</code>	Suprime el aviso de seguridad en genéricos varargs.

3.4 Convenciones de nombres (Java Style Guide)

Elemento	Convención	Ejemplo
Paquetes	minúsculas, sin guiones	<code>com.empresa.rh</code>
Clases / interfaces	PascalCase	<code>Empleado</code> , <code>ListaEnlazada</code>
Métodos / variables	camelCase	<code>calcularTotal()</code> , <code>numeroClientes</code>
Constantes (<code>static final</code>)	MAYUSCULAS_CON_GUION	<code>TAXA_IVA</code>
Genéricos	una letra mayúscula	<code>T</code> , <code>K</code> , <code>V</code> , <code>E</code>

Errores de novato

- Falta y sobra `;` al final de sentencias (a diferencia de C/C++, **no** se ponen tras `{ }` de bloques).
- Olvidar `break` en un `switch` (el *fall-through* sigue ejecutando).
- Comparar `String` con `==` en lugar de `equals()`.
- Nombre de fichero distinto al de la clase pública.

4 Variables, tipos de datos y operadores

4.1 Tipos primitivos (8)

Tipo	Tamaño	Rango / precisión	Por defecto
<code>byte</code>	8 bits	-128 ... 127	0
<code>short</code>	16 bits	-32 768 ... 32 767	0
<code>int</code>	32 bits	$\approx \pm 2.147.483.647 (2^{31}-1)$	0
<code>long</code>	64 bits	$\approx \pm 9.2 \cdot 10^{18}$	0L
<code>float</code>	32 bits	$\approx$ 6-7 decimales	0.0f
<code>double</code>	64 bits	$\approx$ 15-16 decimales	0.0d
<code>char</code>	16 bits	0 ... 65 535 (Unicode, sin signo)	<code>'\u0000'</code>
<code>boolean</code>	~1 bit	<code>true</code> / <code>false</code>	false

Los primitivos NO son objetos

No se les puede llamar un método. Para usarlos como objetos (colecciones, genéricos) existen los *wrappers*: Byte, Short, Integer, Long, Float, Double, Character, Boolean. Java hace **autoboxing** automáticamente (`int` $\leftrightarrow$ `Integer`), pero cada caja **es inmutable** y consume más memoria.

4.2 Declaración y inicialización

```
int edad = 30;           // inicializada
double precio;           // sin inicializar (valor por defecto si es campo)
final int MAX = 100;     // constante: no se puede reasignar
var nombre = "Ana";      // Java 10+: tipo inferido (String)
long gran = 123_456_789L; // sufijo L (usa _ como separador visual)
float f = 3.14f;         // sufijo f obligatorio para float
char c = 'A';            // comillas simples, UN solo carácter
boolean ok = true;
```

Cuándo usar var

Úsalo cuando el tipo es evidente por el lado derecho (`var usuarios = new ArrayList<String>()`). Evítalo con tipos ambiguos (`var x = null`; no compila) o cuando quieras dejar constancia del tipo. **No** es "tipado dinámico": el tipo se fija al declarar.

4.3 Conversiones

```
// Implícita (ampliación, no pierde datos): byte → short → int → long → float → double
int i = 100; long l = i; double d = i;

// Explícita (narrowing): puede perder datos o desbordar
double pi = 3.99;
int entero = (int) pi;           // 3 → se trunca, NO redondea

byte b = (byte) 200;             // -56 → overflow
```

Mangas de overflow

`int.MAX_VALUE + 1` da `-2147483648` sin error. Usa `Math.addExact(a,b)` (lanza excepción) o `long` cuando haya riesgo. En fechas: `Instant.now().toEpochMilli()` con `long`.

4.4 Literales y constantes especiales

```
int dec = 42;
int hex = 0x2A;           // 42
int oct = 052;            // 42
int bin = 0b101010;       // 42
char salto = '\n';        // escapes: \n \t \\ \' \" \r \b \f \uXXXX
String multi = ""
    Texto en varias líneas
    sin escapes molestos.""; // text block (Java 15+)
```

4.5 Operadores

Grupo	Operadores
Aritméticos	<code>+</code> <code>-</code> <code>*</code> <code>/</code> <code>%</code> — la división entre enteros trunca: <code>7/2 == 3</code> . Módulo con negativos conserva el signo del dividendo: <code>-7 % 2 == -1</code>
Incremento	<code>++i</code> (prefijo, primero suma) · <code>i++</code> (sufijo, suma después)
Asignación compuesta	<code>+=</code> <code>-=</code> <code>*=</code> <code>/=</code> <code>%=</code> <code>&=</code> <code> =</code> <code>^=</code> <code><<=</code> <code>>>=</code> (hacen cast implícito)
Comparación	<code>==</code> <code>!=</code> <code><</code> <code><=</code> <code>></code> <code>>=</code> → resultado <code>boolean</code>
Lógicos	<code>&&</code> (AND cortocircuito) · <code> </code> (OR cortocircuito) · <code>!</code> (NOT)
Bit a bit	<code>&</code> <code> </code> <code>^</code> <code>~</code> <code><<</code> <code>>></code> <code>>>></code> — <code>>>></code> desplazamiento sin signo (rellena con 0)
Ternario	<code>cond ? valorSiTrue : valorSiFalse</code>
<code>instanceof</code>	<code>objeto instanceof Tipo</code> → <code>boolean</code> ; con patrón: <code>o instanceof String s && s.length() > 3</code>

Cortocircuito

En `a != null && a.length() > 0`, si `a` es `null` se evalúa `false` sin ejecutar la derecha → evita `NullPointerException`. Ordena siempre las condiciones de más barata/segura a más costosa.

4.6 precedencia (de mayor a menor)

```
1. postfix      a++ a-- () [] .
2. unary        ++a --a ! ~ + - (cast)
3. multiplicative * / %
4. additive     + -
5. shift        << >> >>>
6. relational    < <= > >= instanceof
7. equality      == !=
8. logical AND   &&
9. logical OR    ||
10. ternary      ? :
11. asignación   = += -= ...
```

Cuando dudes, usa paréntesis: el código legible gana siempre a la memorización de la tabla.

5 Control de flujo

5.1 Condicionales

```
if (condicion) {
    // ...
} else if (otra) {
    // ...
} else {
    // ...
}

// Java 21: if con patrones (patrón + bloque de guardia)
if (objeto instanceof String s && s.length() > 5) {
    System.out.println(s.toUpperCase());
}
```

switch (clásico y con patrones, Java 21+)

```
int dia = 3;
switch (dia) {
    case 1, 2, 3 -> System.out.println("Entre semana"); // sin break, sin fall-through
    case 6, 7     -> System.out.println("Fin de semana");
    default       -> System.out.println("Otro");
}

// Switch como expresión (devuelve valor) con yield
String tipo = switch (dia) {
    case 6, 7 -> "SA";
    default   -> yield "LAB";
};

// Switch con patrones (Java 21): sustituye cadenas de if-else/instanceof
static String describir(Object o) {
    return switch (o) {
        case Integer i when i > 0 -> "Entero positivo " + i;
        case String s              -> "Texto: " + s;
        case null                  -> "nulo";
        default                    -> "Desconocido";
    };
}
```

Reglas del switch

Compatible con byte, short, char, int, String, enum y (con patrones) tipos de referencia. Cada rama debe ser inalcanzable en compilación. El `->` anula el *fall-through*; el viejo `case x`: sigue requiriendo `break` (o `yield` en expresiones).

5.2 Bucles

```
// for clásico
for (int i = 0; i < 10; i++) { /* ... */ }

// for-each (recorre arrays y Iterable)
for (String s : lista) {
    System.out.println(s);
}

// while: se evalúa ANTES de cada vuelta
while (condicion) { /* ... */ }

// do-while: se ejecuta AL MENOS una vez
do { /* ... */ } while (condicion);

// Etiquetas (labels): útil para salir de bucles anidados
externo:
for (int i = 0; i < 3; i++)
    for (int j = 0; j < 3; j++)
        if (j == 1) break externo;    // sale de LOS DOS bucles
```

Sentencia	Efecto
<code>break</code>	Abandona el bucle/switch más externo
<code>continue</code>	Pasa a la siguiente iteración
<code>return</code>	Abandona el método (devolviendo valor si lo hay)
<code>yield</code>	Devuelve un valor desde una rama de un <code>switch</code> -expresión

Bucle infinito típico

`for (int i = 0; i < n; i++)` con `n <= 0` no entra; pero `while (true)` sin `break` cuelga el programa. Si incrementas un byte hasta 127, `++b` pasa a -128 y el bucle nunca termina.

6 Arrays y Strings

6.1 Arrays

```
// Declaración e inicialización
int[] nums    = new int[5];           // 0,0,0,0,0
int[] nums2   = {1, 2, 3, 4};
String[] nombres = new String[3];    // null,null,null

int primero = nums2[0];
nums2[0] = 99;
int n = nums2.length;                // propiedad (no es método): 4

// Array multidimensional (en realidad: array de arrays)
int[][] m = {
    {1, 2, 3},
    {4, 5, 6}
};
int v = m[1][2];                     // 6
m[0] = new int[10];                 // Las "filas" pueden tener distinta longitud
```

ArrayIndexOutOfBoundsException

Los índices van de **0 a length-1**. Recorre siempre con `i < array.length`, nunca `<=`. Comprueba límites antes de acceder a posiciones calculadas.

```
// Arrays.sort, binarySearch, Arrays.equals, Arrays.fill, Arrays.copyOf
Arrays.sort(datos);
int pos = Arrays.binarySearch(datos, 42); // requiere array ordenado
System.out.println(Arrays.toString(datos)); // "[1, 2, 3]"
```

6.2 String: inmutabilidad

`String` es **immutable**: cada operación crea un objeto nuevo. Por eso, concatenar en bucle con `+=` es $O(n^2)$ y genera basura; usa `StringBuilder`.

```
String s = "Hola";
s.concat("!"); // NO modifica s: devuelve "Hola!" (que ignoramos)
String t = s + " mundo";
String u = s.toUpperCase(); // "HOLA"
```

Métodos imprescindibles de String

Método	Descripción
<code>length()</code>	Número de caracteres (UTF-16 code units)
<code>isEmpty()</code> / <code>isBlank()</code>	Cadena vacía / cadena con solo espacios (Java 11+)
<code>charAt(i)</code> / <code>chars()</code>	Carácter en posición <code>i</code> / IntStream de códigos
<code>indexOf(x)</code> / <code>lastIndexOf(x)</code>	Posición (o -1 si no está)
<code>substring(a, b)</code>	Desde <code>a</code> hasta <code>b-1</code> (fin exclusivo)
<code>equals(o)</code> / <code>equalsIgnoreCase(o)</code>	Comparación de contenido (usa SIEMPRE estos)
<code>compareTo(o)</code>	Orden lexicográfico: <0, 0, >0 (implementa Comparable)
<code>startsWith(p)</code> / <code>endsWith(p)</code>	Comprueba prefijo/sufijo
<code>replace(a, b)</code> / <code>replaceAll(regex, r)</code>	Sustituye literal / por expresión regular
<code>split(regex)</code>	Devuelve <code>String[]</code>
<code>trim()</code> / <code>strip()</code>	Quita espacios (strip respeta Unicode, Java 11+)
<code>toUpperCase()</code> / <code>toLowerCase()</code>	Mayúsculas/minúsculas (cuidado con locales: <code>toLowerCase(Locale.ROOT)</code>)
<code>contains(x)</code>	Devuelve <code>true</code> si contiene la subcadena
<code>formatted(...)</code> / <code>String.format</code>	Formato estilo C: <code>%s %d %.2f %n</code>
<code>join(sep, elems)</code>	Une elementos: <code>String.join(", ", lista)</code>
<code>repeat(n)</code>	Repite (Java 11+)

```
String correo = "ana@ejemplo.com";
correo.contains("@");           // true
correo.split("@")[0];          // "ana"
"precio: %s €".formatted(19.99); // "precio: 19.99 €"
String.join(" - ", "a", "b", "c"); // "a - b - c"
```

¿== o equals()?

`==` compara **referencias** (¿apuntan al mismo objeto?), `equals()` compara **contenido**. Solo usan `==` con primitivos o para comprobar null. Bonus: `Objects.equals(a, b)` es null-safe.

StringBuilder / StringBuffer

```
StringBuilder sb = new StringBuilder(); // NO sincronizado (usa este)
sb.append("Hola").append(" ").append(2026);
sb.insert(0, "> ");
sb.replace(0, 2, "");
sb.reverse();
sb.deleteCharAt(0);
String fin = sb.toString();

// StringBuffer = mismo API pero sincronizado (threads) → más lento
```

6.3 Comparar y analizar números

```
Integer.parseInt("42"); // String → int (Lanza NumberFormatException)
Integer.valueOf("42"); // String → Integer (usa caché -128..127)
Long.parseLong("99999999");
Double.parseDouble("3.14");
42 + ""; // int → String (no recomendado)
String.valueOf(42); // forma idiomática
String.valueOf(42) // 42
```


7 Orientación a objetos: clases y objetos

Una **clase** es el molde (plantilla); un **objeto** es una instancia concreta. La clase define *atributos* (estado) y *métodos* (comportamiento).

7.1 Definición completa de una clase

```
public class CuentaBancaria {

    // --- Atributos (estado) ---
    private final String iban;      // inmutable, solo se asigna en el constructor
    private double saldo;
    private static int totalCuentas; // compartido por TODA la clase

    // --- Constructores ---
    public CuentaBancaria(String iban) {
        this.iban = Objects.requireNonNull(iban, "IBAN obligatorio");
    }

    public CuentaBancaria(String iban, double saldo) {
        this(iban); // delega en otro constructor (DEBE ser la 1ª línea)
        depositar(saldo);
    }

    // --- Métodos de instancia ---
    public void depositar(double cantidad) {
        if (cantidad <= 0) throw new IllegalArgumentException("Cantidad > 0");
        saldo += cantidad;
    }

    public double getSaldo() { return saldo; }

    // --- Métodos estáticos (de clase) ---
    public static int getTotalCuentas() { return totalCuentas; }

    // --- toString / equals / hashCode ---
    @Override
    public String toString() {
        return "CuentaBancaria{iban='" + iban + "', saldo=" + saldo + '}';
    }

    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (o == null || getClass() != o.getClass()) return false;
        CuentaBancaria esa = (CuentaBancaria) o;
        return iban.equals(esa.iban);
    }

    @Override
    public int hashCode() { return iban.hashCode(); }
}
```

7.2 Crear y usar objetos

```
CuentaBancaria c1 = new CuentaBancaria("ES91 2100 0418 4502 0005 1332");
c1.depositar(500);
System.out.println(c1.getSaldo()); // usa toString() si lo pones directamente

CuentaBancaria c2 = c1; // c2 y c1 apuntan al MISMO objeto
c2.depositar(10); // también cambia el saldo de c1
```

Paso de parámetros

Java pasa **siempre por valor**: se copia la *referencia*. Dentro del método puedes mutar el objeto apuntado (se ve desde fuera) o reasignar el parámetro (no se ve fuera).

7.3 Encapsulamiento

Modificador	Clase	Paquete	Subclase	Todos
<code>public</code>	✓	✓	✓	✓
<code>protected</code>	✓	✓	✓	✗
(sin modificador)	✓	✓	✗	✗
<code>private</code>	✓	✗	✗	✗

- Atributos `private` + acceso mediante *getters/setters* (o `record` si es solo lectura).
- Los **campos** `static` pertenecen a la clase, no a la instancia. Inicialización en *bloque estático*.
- **Constantes**: `public static final double IVA = 0.21;`

```
public class Config {
    public static final String VERSION = "2.1.0";
    private static final Map<String, String> PROPS = new HashMap<>();

    static { // bloque estático: se ejecuta al cargar la clase
        PROPS.put("app.nombre", "Demo");
    }
}
```

7.4 Sobreloads y sobrescritura

	Sobrecarga (<i>overload</i>)	Sobrescritura (<i>override</i>)
Cuándo	Mismo nombre, distinta firma , en la MISMA clase	Mismo nombre y firma en una SUBclase
Decide	El compilador (resolución estática)	La JVM en ejecución (dinámico)
Requisito	Distinto número/tipo de parámetros	Firma idéntica; visibilidad $\geq$ (no más restrictiva); no lanza checked más amplias

```

void imprimir(String s) { }
void imprimir(int i)    { }           // sobrecarga

class Gato extends Animal {
    @Override public void sonar() { System.out.println("Miau"); }
}

```

7.5 Objetos, equals y hashCode (contrato)

1. **Reflexivo:** `a.equals(a)` → true.
2. **Simétrico:** `a.equals(b)` ⇔ `b.equals(a)`.
3. **Transitivo** y consistente.
4. Sobre `null`: `a.equals(null)` → false.
5. Si redefines `equals`, redefine también `hashCode` → si no, fallan `HashMap` / `HashSet`.

Truco para hashCode

`return Objects.hash(iban, saldo);` — y en equals: `getClass() != o.getClass()` (o usa `instanceof` si permites herencia).

7.6 Clases y bloques

```

// Bloque de instancia: se ejecuta en cada new, antes del constructor
{
    registros.add("nuevo");
}

// Clases anidadas
class Exterior {
    private static class Estatica { }           // anidada estática
    class Miembro { }                          // miembro interna (accede a Exterior.this)
    void metodo() {
        class Local { }                       // clase local
        new Runnable() {                     // clase anónima (sin nombre)
            public void run() { }
        };
    }
}

```

8 Herencia, polimorfismo, abstracción e interfaces

8.1 Herencia (extends)

```
public class Animal {
    protected String nombre;
    public Animal(String nombre) { this.nombre = nombre; }
    public void sonar() { System.out.println("..."); }
    public void comer() { System.out.println(nombre + " come"); }
}

public class Perro extends Animal {
    public Perro(String nombre) {
        super(nombre); // DEBE ser la primera sentencia: Llama al constructor padre
    }
    @Override
    public void sonar() { System.out.println("Guau"); }
    public void traer() { System.out.println("Trae la pelota"); }
}
```

- **Java no permite herencia múltiple de clases** (un solo `extends`), pero sí múltiple implementación de interfaces.
- `super()` → constructor padre; `super.metodo()` → versión del padre (útil al sobrescribir).
- `this` → objeto actual; debe ser la 1ª línea si llama a otro constructor propio.
- La clase `Object` es la raíz de todo: `toString`, `equals`, `hashCode`, `getClass`, `clone`, `wait`, `notify`.
- Todos los campos se reservan al nacer el objeto: **slicing** = al asignar un objeto hijo a una referencia padre se "pierde" el acceso al resto de miembros (el objeto sigue completo).

8.2 Polimorfismo

```
Animal a = new Perro("Rex");
a.sonar(); // "Guau" → se ejecuta la versión de Perro (dispatch dinámico)
// a.traer(); // NO compila: el compilador mira el tipo de la REFERENCIA (Animal)

Animal[] zoo = { new Perro("Rex"), new Gato("Mishi") };
for (Animal x : zoo) x.sonar(); // cada uno responde a su manera

// Downcasting con comprobación SIEMPRE seguro
if (a instanceof Perro p) {
    p.traer(); // Java 16+: pattern matching ya convierte
}
```

ClassCastException

El casteo sin comprobar falla en tiempo de ejecución. Regla de oro: `instanceof` antes de castear, o usa el patrón `instanceof Tipo t`.

8.3 Clases abstractas

```
public abstract class Forma {
    public abstract double area();           // sin cuerpo: obliga a las subclases
    public void describir() {                // método concreto: se hereda tal cual
        System.out.println("Área = " + area());
    }
}

public class Circulo extends Forma {
    private final double r;
    public Circulo(double r) { this.r = r; }
    @Override public double area() { return Math.PI * r * r; }
}

// new Forma(); → ERROR: no se puede instanciar una abstracta
```

Abstract vs interfaz: usa clase abstracta cuando hay *estado o código compartido* de la misma jerarquía; usa interfaz cuando defines una *capacidad/contrato* que clases de líneas distintas deben cumplir.

8.4 Interfaces

```
public interface Volable {
    int VELOCIDAD_MAX = 100;                // implícito: public static final
    void volar();                           // implícito: public abstract
    default void aterrizar() {              // método por defecto (Java 8+)
        System.out.println("Aterrizando");
    }
    static boolean esValida(Volable v) { return v != null; }
}

public interface Nadable { void nadar(); }

public class Pato implements Volable, Nadable { // múltiple implementación ✓
    @Override public void volar() { /* ... */ }
    @Override public void nadar() { /* ... */ }
}
```

Reglas clave

- Todos los campos son `public static final` (constantes).
- Desde Java 8: `default` (heredable) y `static`. Desde Java 9: métodos `private` auxiliares.
- **Conflictos:** si dos interfaces aportan el mismo `default`, la clase DEBE sobrescribirlo y elegir con `Interfaz.super.metodo()`.
- Una clase que no implementa todo lo abstracto debe ser `abstract`.
- **Herencia de firmas** (Java 8+): `interface A extends B, C`.

```
public interface A { default void hola() { } }
public interface B { default void hola() { } }
public class C implements A, B {
    @Override public void hola() {
        A.super.hola();           // elección explícita obligatoria
    }
}
```

8.5 Interfaces vs clases abstractas

	Interfaz	Clase abstracta
Múltiple	Sí (<code>implements</code> varias)	No (un solo <code>extends</code>)
Estado	No (solo constantes)	Sí (atributos)
Constructores	No	Sí
Implementación	<code>default</code> / <code>static</code> / <code>private</code>	Cualquier método con cuerpo
Cambio	Añadir <code>default</code> = compatible	Añadir <code>abstract</code> = rompe subclases

9 Enums, records y SOLID

9.1 Enums

```
public enum Prioridad {
    BAJA(1), MEDIA(2), ALTA(3), CRITICA(4);    // cada valor es una CONSTANTE SINGLETON

    private final int peso;

    Prioridad(int peso) { this.peso = peso; }    // constructor implícito private

    public int getPeso() { return peso; }

    public static Prioridad desdePeso(int p) {
        for (Prioridad pr : values())
            if (pr.peso == p) return pr;
        throw new IllegalArgumentException("peso " + p);
    }
}

// Uso
Prioridad p = Prioridad.ALTA;
Prioridad[] todos = Prioridad.values();
Prioridad v = Prioridad.valueOf("ALTA");    // mayúsculas exactas
p.ordinal();    // 2 (posición) → evítalo en lógica

switch (p) {
    case BAJA, MEDIA -> System.out.println("Normal");
    case ALTA, CRITICA -> System.out.println("Urgente");
}
```

También pueden **implementar interfaces** y tener métodos abstractos por constante (estrategia tipo "constant-specific body").

9.2 Records (Java 16+)

```
public record Persona(String nombre, int edad) {
    // Compact constructor: validación antes de asignar
    public Persona {
        Objects.requireNonNull(nombre);
        if (edad < 0) throw new IllegalArgumentException("edad negativa");
    }
    public boolean esMayor() { return edad >= 18; }
}

Persona p = new Persona("Luis", 30);
p.nombre();    // accessors (no getNombre())
p.toString();    // "Persona[nombre=Luis, edad=30]"
p.equals(other);    // basado en todos los componentes
```

- **Immutable** y `final`: no extiende otras clases (sí implementa interfaces) e implícita `extends Record → Object`.
- Genera: constructor, accessors, `toString`, `equals`, `hashCode`.

- Sin setters: para un campo derivado usa un método (`p.esMayor()`).

9.3 Clases funcionales modernas

- **sealed** (Java 17): controla quién puede extender.

```
public sealed interface Shape permits Circle, Square { }
public record Circle(double r) implements Shape { }
public non-sealed interface Square implements Shape { } // abre otra rama
```

- **pattern matching** en `instanceof` y en `switch` (ver §5.1).

9.4 Principios SOLID

Principio	Idea	Ejemplo en Java
Single Responsibility	Una clase, una razón para cambiar	Separa <code>UsuarioDAO</code> de <code>UsuarioEmail</code>
Open/Closed	Abierto a extensión, cerrado a modificación	Añadir una <code>Impresion</code> sin tocar el código existente
Liskov	Las subclases deben sustituir al padre sin romper nada	No lanzar <code>UnsupportedOperationException</code> en métodos heredados
Interface Segregation	Interfaces pequeñas y específicas	<code>Lectura</code> / <code>Escritura</code> en vez de una <code>Todo</code>
Dependency Inversion	Depende de abstracciones, no de concretos	Constructor con <code>Repositorio</code> , no con <code>JdbcRepositorio</code>

10 Manejo de excepciones

10.1 Jerarquía

```
java.lang.Throwable
├── Error                                → fallos graves de la JVM (OutOfMemoryError, StackOverflowError)
│                                       No se capturan: no hay nada que hacer.
└── Exception
    ├── Checked (comprobadas en compilación)
    │   ├── IOException, SQLException, FileNotFoundException
    │   └── Exception personalizadas que extienden Exception
    └── Unchecked (RuntimeException) → errores de programación
        ├── NullPointerException, ArrayIndexOutOfBoundsException
        ├── ClassCastException, IllegalArgumentException
        ├── IllegalStateException, ArithmeticException
        └── NumberFormatException, ConcurrentModificationException
```

10.2 try / catch / finally

```
try {
    int r = 10 / divisor;
} catch (ArithmeticException e) {
    System.err.println("División por cero: " + e.getMessage());
} catch (RuntimeException e) {      // de MÁS ESPECÍFICA a MÁS GENÉRICA
    log.error("error inesperado", e);
} finally {
    // SIEMPRE se ejecuta (aunque haya return). Ideal para liberar recursos.
}
```

- `throw new Excepcion("motivo")` lanza; `throws Excepcion` en la firma delega al llamador.
- `return` dentro de `try` no impide ejecutar `finally`.
- Nunca un `catch` vacío: al menos registra el error.

10.3 try-with-resources (Java 7+)

```
// Los recursos (AutoCloseable) se cierran SOLOS en orden inverso
try (BufferedReader br = new BufferedReader(new FileReader("datos.txt"));
    Connection con = DriverManager.getConnection(url, user, pass)) {

    String linea;
    while ((linea = br.readLine()) != null) {
        System.out.println(linea);
    }
} catch (IOException | SQLException e) {    // multi-catch: misma variable
    Logger.getLogger().error("fallo", e);
}
```

Regla

Todo recurso que implementa `AutoCloseable` (streams, conexiones, canales) va en la cabecera del `try`. Sustituye a los `finally` manuales con `close()` olvidadizo.

10.4 Crear excepciones propias

```
public class SaldoInsuficienteException extends Exception { // checked
    private final double faltante;
    public SaldoInsuficienteException(double faltante) {
        super("Faltan " + faltante + " €");
        this.faltante = faltante;
    }
    public double getFaltante() { return faltante; }
}

// Uso
if (saldo < cantidad)
    throw new SaldoInsuficienteException(cantidad - saldo);
```

Para errores de programación/validación usa una **unchecked**: `extends RuntimeException` (p. ej. `DatosInvalidosException`). Evita excepciones genéricas como `Exception` o `RuntimeException` en firmas.

10.5 Buenas prácticas

Evita esto

- `catch (Exception e) {}` → traga el fallo y dificulta el debug.
- Usar excepciones para control de flujo normal (son lentas: crean traza).
- Devolver `null` donde podría haber `Optional`.
- Lanzar y capturar la misma excepción en el mismo método.

```
// Patrón de captura-re-lanzar conservando la causa original
try { ... }
catch (IOException e) {
    throw new RepositoriesException("Error leyendo fichero", e); // causa = e
}
```

10.6 Stack trace y depuración

```
e.printStackTrace(); // rápido, solo en desarrollo
Logger.getLogger().error("contexto", e); // forma correcta en producción
Objects.requireNonNull(x, "x es obligatorio"); // lanza NPE con mensaje útil
```

Lee la traza de abajo hacia arriba: la **primera línea** de tu paquete es el origen real del problema.

10.7 Optional (Java 8+)

```
Optional<String> o = Optional.ofNullable(datos);
String v = o.orElse("defecto");
String v2 = o.orElseGet(() -> calcular()); // Lazy
o.ifPresent(s -> System.out.println(s));
o.map(String::length).orElse(0);
String v3 = o.orElseThrow(() -> new IllegalStateException("sin dato"));
```

Cuándo usar Optional

Como **retorno de métodos** que pueden no tener valor (y evitar null). Nunca como parámetro, ni como campo de clase, ni dentro de colecciones.

11 Colecciones (Collection Framework)

11.1 Estructura general

```
Iterable<E>                                → permite for-each
├── Collection<E>
│   ├── List<E>                            ordenado, con duplicados, índice
│   │   ├── ArrayList                      acceso aleatorio  $O(1)$ , borrar en medio  $O(n)$ 
│   │   ├── LinkedList                    inserción/borrado  $O(1)$  si tienes el nodo, Deque
│   │   └── Vector                        sincronizado, obsoleto (usar Collections.synchronizedList)
│   ├── Set<E>                            sin duplicados
│   │   ├── HashSet                      árbol hash →  $O(1)$ , sin orden
│   │   ├── LinkedHashSet                insertion order
│   │   └── TreeSet                      ordenado (SortedSet) →  $O(\log n)$ 
│   ├── Queue/Deque                       FIFO / pila
│   │   ├── ArrayDeque                   pila/cola rápido (usa este)
│   │   ├── PriorityQueue                heap (el menor primero)
│   │   └── LinkedList
└── Map<K,V>                             (no extiende Collection, pero es parte del framework)
    ├── HashMap                           $O(1)$  medio, sin orden, claves null
    ├── LinkedHashMap                    orden de inserción / acceso
    ├── TreeMap                          orden por clave (SortedMap) →  $O(\log n)$ 
    ├── Hashtable                        sincronizado, obsoleto
    ├── ConcurrentHashMap                multihilo (ver §15)
    └── Properties                       Hashtable<String,String> para ficheros .properties
```

11.2 List

```
List<String> lista = new ArrayList<>();
lista.add("Java");
lista.add(1, "Kotlin");           // en posición
lista.get(0);                     //  $O(1)$ 
lista.set(0, "Java 21");
lista.remove("Kotlin");           // por objeto (busca y borra)
lista.remove(0);                  // por índice → devuelve el elemento
lista.contains("Java");
lista.indexOf("Java");
lista.size();                     // método, no propiedad
lista.isEmpty();
lista.clear();
lista.sort(Comparator.reverseOrder());
lista.addAll(otra);
lista.retainAll(otra);            // intersección

// Lista inmutable / copias
List<String> ro = List.of("a", "b"); // immutable, sin null
List<String> copia = new ArrayList<>(lista); // copia defensiva
List<String> unmodifiable = List.copyOf(lista);
```

11.3 Set

```
Set<String> unicos = new HashSet<>();
unicos.add("a"); unicos.add("a"); // add devuelve false, sigue habiendo 1
unicos.contains("a");

Set<String> orden = new TreeSet<>(lista); // ordena alfabéticamente
Set<String> union = new HashSet<>(a); union.addAll(b);
Set<String> inter = new HashSet<>(a); inter.retainAll(b);
Set<String> dif = new HashSet<>(a); dif.removeAll(b);
```

Regla de oro

Si usas la clase como clave de un HashMap o miembro de un HashSet, **debes sobrescribir equals y hashCode**. Como clave usa tipos inmutables (String, Integer, records).

11.4 Map

```
Map<String, Integer> edades = new HashMap<>();
edades.put("Ana", 31);
edades.put("Luis", 27);
edades.get("Ana"); // 31, o null si no existe
edades.getOrDefault("Bob", 0); // 0
edades.containsKey("Ana");
edades.containsValue(31);
edades.remove("Luis");
edades.size();

// getOrDefault + merge = contador idiomático
for (String p : palabras) {
    conteo.put(p, conteo.getOrDefault(p, 0) + 1);
}
conteo.merge(palabra, 1, Integer::sum);

// Iterar
for (Map.Entry<String, Integer> e : edades.entrySet()) {
    System.out.println(e.getKey() + " → " + e.getValue());
}
edades.forEach((k, v) -> System.out.println(k + "=" + v));

// API funcional
edades.putIfAbsent("Ana", 0);
edades.compute("Ana", (k, v) -> v == null ? 1 : v + 1);
edades.computeIfAbsent("Bob", k -> new ArrayList<>());
edades.replace("Ana", 32);

// Map inmutable (Java 9+)
Map<String, Integer> fijo = Map.of("a", 1, "b", 2);
Map<String, Integer> nulo = Map.ofEntries(Map.entry("a", 1));
```

11.5 Queue / Deque

```
Queue<String> cola = new ArrayDeque<>();
cola.offer("a");           // añade (false si no puede)
cola.add("b");             // añade (lanza excepción si falla)
cola.peek();              // cabeza sin quitar (null si vacía)
cola.poll();              // extrae y elimina (null si vacía)

Deque<String> pila = new ArrayDeque<>();
pila.push("x"); pila.push("y"); // pila (LIFO)
String cima = pila.pop();
pila.isEmpty();
```

11.6 Iteración y modificaciones concurrentes

```
// ❌ ConcurrentModificationException: modificar durante el for-each
for (String s : lista) if (s.isEmpty()) lista.remove(s);

// ✅ Opción A: Iterator.remove
Iterator<String> it = lista.iterator();
while (it.hasNext()) if (it.next().isEmpty()) it.remove();

// ✅ Opción B: removeIf (recomendada)
lista.removeIf(String::isEmpty);

// ✅ Opción C: copia
for (String s : new ArrayList<>(lista)) { ... }
```

11.7 Utilidades Collections / Arrays

```
Collections.sort(l); // in-place
Collections.sort(l, Comparator.comparing(Persona::edad));
Collections.reverse(l);
Collections.shuffle(l); // aleatorio
Collections.max(l); Collections.min(l);
Collections.frequency(l, "a"); // cuántas veces aparece
Collections.unmodifiableList(l); // vista de solo lectura
Collections.synchronizedList(l); // wrapper sincronizado
Collections.emptyList(); Collections.singleton("x");
Collections.unmodifiableMap(m);

Arrays.sort(arr); Arrays.parallelSort(arr); Arrays.stream(arr);
```

11.8 ¿Qué colección elijo?

Necesidad	Colección
Acceso por índice rápido, lo más usado	ArrayList
Cola FIFO / pila LIFO	ArrayDeque
Elementos únicos sin orden	HashSet
Elementos únicos ordenados / rangos	TreeSet / TreeMap
Clave-valor, la mayoría de casos	HashMap
Acceso secuencial por nodo, poco común	LinkedList
Múltiples hilos escribiendo a la vez	ConcurrentHashMap / CopyOnWriteArrayList
Orden de inserción	LinkedHashMap / LinkedHashSet

12 Genéricos

Los genéricos dan **type safety en compilación** y evitan `ClassCastException` en ejecución sin hacer casting manual.

12.1 Genéricos de clases y métodos

```
public class Caja<T> {
    private T contenido;
    public void poner(T c)      { this.contenido = c; }
    public T obtener()         { return contenido; }
}

Caja<String> caja = new Caja<>(); // diamond <> infiere desde Java 7
caja.poner("texto");
String s = caja.obtener();        // sin cast

// Método genérico: el tipo se deduce de los argumentos
public static <T> List<T> listaDe(T... elems) {
    return new ArrayList<>(Arrays.asList(elems));
}

List<Integer> l = listaDe(1, 2, 3); // T = Integer
```

12.2 Bounded type parameters y wildcards

```
// Extensión: T es Animal o una subclase
public static <T extends Comparable<T>> T maximo(List<T> l) { ... }

public <T extends Number & Serializable> void f(T x) { } // multi-límite (una clase + N interfaces)

// Wildcards
void imprimir(List<?> l) // PECS-ish: cualquier tipo (lectura)
void sumar(List<? extends Number> l) // Producer: sólo LEER Number (covariante)
void anadir(List<? super Integer> l) // Consumer: sólo ESCRIBIR Integer (contravariante)
```

Regla PECS

Producer ? extends T (sacas datos) · **E**xsumer ? super T (metes datos) · **C**ollection Collection<T> · **S**hift (no aplica). List<String> **no** es subtipo de List<Object>: los genéricos son *invariantes*.

12.3 Erasure y sus límites

- El compilador **borra** los tipos en tiempo de ejecución (compatibilidad con Java 5): no puedes hacer `new T()`, `T.class`, `instanceof List<String>` ni sobrecargar `f(List<String>)` y `f(List<Integer>)` (misma firma tras borrar).
- Soluciones: pasar `Class<T>`, `Supplier<T>` o un `TypeToken` (Gson/Jackson).


```
public T crear(Class<T> tipo) throws InstantiationException, IllegalAccessException {  
    return tipo.getDeclaredConstructor().newInstance();  
}
```

12.4 Genéricos avanzados (PECS en la práctica)

```
// Collections.copy(destino, origen) → Collection<? super T> destino, Collection<? extends T>  
    origen  
// Comparator.reversed(), Comparator.thenComparing(...) → uso natural de wildcards  
// Arrays.asList(T...) → lista sobre el array original (¡cuidado con modificaciones!)
```

13 Lambdas, funciones funcionales y Streams

13.1 Interfaces funcionales

Interfaz con **un único método abstracto** (SAM) → apta para lambda y method reference. Se anota con `@FunctionalInterface`.

Interfaz	Método	Qué recibe / devuelve	Ejemplo
<code>Function<T,R></code>	<code>R apply(T)</code>	transforma $T \rightarrow R$	<code>s -> s.length()</code>
<code>Predicate<T></code>	<code>boolean test(T)</code>	filtro booleano	<code>s -> !s.isEmpty()</code>
<code>Consumer<T></code>	<code>void accept(T)</code>	acción sin retorno	<code>s -> print(s)</code>
<code>Supplier<T></code>	<code>T get()</code>	produce valor	<code>() -> new Random()</code>
<code>BiFunction<T,U,R></code>	<code>R apply(T,U)</code>	dos entradas	<code>(a,b) -> a + b</code>
<code>UnaryOperator<T></code>	<code>T apply(T)</code>	$T \rightarrow T$	<code>s -> s.trim()</code>
<code>Runnable</code>	<code>run()</code>	sin args ni retorno	<code>() -> System.out.print("x")</code>
<code>Callable<V></code>	<code>V call()</code>	puede lanzar excepciones	<code>() -> descargar()</code>

13.2 Sintaxis de lambda

```
(String a, int b) -> { return a.length() + b; } // completo
a -> a.toUpperCase() // un paréntesis omitible
() -> calcular() // sin parámetros
(a, b) -> a + b // implícito (tipos deducidos)
(x) -> x * x // un solo paréntesis permitido

// Expresión vs bloque
n -> n % 2 == 0 // expresión: devuelve el valor (implícito return)
n -> { log(n); return n > 0; } // bloque: return OBLIGATORIO
```

- **Final effectively**: las variables capturadas deben ser effectively final (no reasignadas).
- Las lambdas **no** crean ámbito propio: `this` = la clase exterior.
- Una lambda es *sintaxis*; para guardarla en variable usa el tipo de interfaz funcional.

13.3 Method references (referencias a métodos)

```
System.out::println // bound instance: s -> System.out.println(s)
String::valueOf // static: x -> String.valueOf(x)
String::length // unbound: s -> s.length()
lista::get // instance: i -> lista.get(i)
this::metodoPropio
Persona::new // constructor
```

13.4 Streams: la API funcional

Un **Stream** es una secuencia de elementos *soportada por una fuente*, con operaciones *intermedias* (lazy) y una *terminal* que dispara la ejecución. **No almacena datos** y normalmente no se puede recorrer dos veces.

```
Stream<String> st = lista.stream();           // desde colección
Stream<int[]> st2 = Stream.of(arr);           // desde array (Stream.of)
Stream<String> st3 = Stream.of("a", "b");
Stream<String> st4 = Stream.generate(() -> "x").limit(5);
Stream<Integer> st5 = Stream.iterate(0, n -> n + 2).limit(10);
IntStream nums = IntStream.rangeClosed(1, 10);
Stream<String> lineas = Files.lines(Path.of("f.txt"));
```

Operaciones intermedias (lazy, devuelven Stream)

Operación	Descripción
<code>filter(Predicate)</code>	Deja solo los que cumplen
<code>map(Function)</code>	Transforma 1 a 1
<code>flatMap(Function)</code>	Transforma y "aplana" streams anidados
<code>distinct()</code>	Sin duplicados (usa equals/hashCode)
<code>sorted()</code> / <code>sorted(cmp)</code>	Ordena (natural o con comparador)
<code>peek(Consumer)</code>	Observa sin modificar (debug)
<code>limit(n)</code> / <code>skip(n)</code>	Trunca / salta
<code>takeWhile(p)</code> / <code>dropWhile(p)</code>	Corta mientras cumple (Java 9+)
<code>mapToInt/ToLong/ToDouble</code>	Streams primitivos (evitan boxing)

Operaciones terminales

Terminal	Resultado
<code>forEach / forEachOrdered</code>	Acción por elemento
<code>collect(Collector)</code>	Agrupa: <code>toList()</code> , <code>toSet()</code> , <code>joining()</code> , <code>groupingBy()</code> , <code>partitioningBy()</code> , <code>counting()</code> , <code>toMap()</code>
<code>toList()</code>	Lista inmutable rápida (Java 16+)
<code>reduce(identity, BinaryOperator)</code>	Acumula a un único valor
<code>count()</code>	Número de elementos
<code>min(cmp) / max(cmp)</code>	<code>Optional<T></code>
<code>anyMatch / allMatch / noneMatch</code>	<code>boolean</code> (cortocircuitan)
<code>findFirst / findAny</code>	<code>Optional<T></code>
<code>toArray()</code>	Array de Object o con fábrica
<code>iterator() / spliterator()</code>	Compatibilidad imperativa

13.5 Ejemplos completos

```
record Persona(String nombre, int edad, String ciudad) {}

// 1. Filtrar + transformar + recoger
List<String> mayores = personas.stream()
    .filter(p -> p.edad() >= 18)
    .map(Persona::nombre)
    .sorted()
    .toList();

// 2. Agrupar por ciudad y contar
Map<String, Long> porCiudad = personas.stream()
    .collect(Collectors.groupingBy(Persona::ciudad, Collectors::counting));

// 3. Media de edades
double media = personas.stream()
    .mapToInt(Persona::edad)
    .average()
    .orElse(0);

// 4. Unir en un String separado por comas
String csv = personas.stream()
    .map(Persona::nombre)
    .collect(Collectors.joining(", "));

// 5. Map nombre -> edad (cuidado con claves duplicadas: mergeFunction)
Map<String, int> idx = personas.stream()
    .collect(Collectors.toMap(Persona::nombre, Persona::edad, (a, b) -> a));

// 6. Partitioning (cumple / no cumple)
Map<Boolean, List<Persona>> part = personas.stream()
    .collect(Collectors.partitioningBy(p -> p.edad() > 30));

// 7. Reduce: suma
int total = numeros.stream().reduce(0, Integer::sum);

// 8. Texto -> palabras únicas ordenadas
List<String> palabras = texto.toLowerCase()
    .split("[^a-záéíóúñ]+")    // texto.split es String[]
    , []);
```

Cuidado con toList()

`Collectors.toList()` devuelve una lista **mutable** (`ArrayList`); `Stream.toList()` (Java 16+) devuelve una **inmutable**. Ambas rechazan null si el stream permite filtrar, y `stream.toList()` lanza `NullPointerException` si hay null.

13.6 Streams paralelos

```
List<String> r = lista.parallelStream()
    .filter(s -> !s.isBlank())
    .map(String::toUpperCase)
    .collect(Collectors.toList());    // ForkJoinPool común
```

-  Útil con **muchos datos** y operaciones *stateless/independientes* (map/filter). `forEachOrdered` serializa el orden.
-  Evítalo con operaciones dependientes del orden (`limit` temprano), efectos laterales, I/O, o listas pequeñas (overhead > beneficio).
- Para E/S paralela controlada usa `ExecutorService` (§15).

13.7 Optional con Streams

```
Optional<String> o = lista.stream()
    .filter(s -> s.startsWith("J"))
    .findFirst();
o.ifPresent(System.out::println);
List<String> todos = lista.stream()
    .filter(s -> s.length() > 3)
    .toList();
```

14 Entrada/Salida y ficheros

14.1 Jerarquía

Paquete	Modelo	Uso
<code>java.io</code>	Streams (bytes) y Readers/Writers (caracteres)	Herencia clásica: <code>InputStream</code> , <code>Reader</code>
<code>java.nio</code> / <code>java.nio.file</code>	Buffered/Canales y Path	Moderno, más rápido y con API funcional

14.2 Leer y escribir ficheros (forma recomendada, Java 11+)

```
Path p = Path.of("datos", "inventario.txt");

// Lectura completa a String / a líneas / a Stream
String contenido = Files.readString(p, StandardCharsets.UTF_8);
List<String> lineas = Files.readAllLines(p);

try (Stream<String> s = Files.lines(p)) {
    s.filter(l -> l.contains("ERROR"))
      .forEach(System.out::println);
}

// Escritura
Files.writeString(p, "Hola mundo\n", StandardOpenOption.APPEND);
Files.write(p, lineas); // List<String> → una línea por elemento

// Operaciones
Files.exists(p);      Files.isDirectory(p);  Files.size(p);
Files.createDirectories(Path.of("salida/sub"));
Files.copy(origen, destino); // sin REPLACE_EXISTING lanza FileAlreadyExists
Files.move(origen, destino, StandardCopyOption.REPLACE_EXISTING);
Files.deleteIfExists(p);
Files.walk(dir).filter(Files::isRegularFile).forEach(System.out::println);
Files.walk(dir).forEach(x -> Files.delete(x)); // borrar árbol
Files.list(dir).forEach(System.out::println); // Stream<Path>: cierra con try-with-resources
```

14.3 Streams de bytes (clásico)

```
// Escritura con buffer (buena práctica: envuelve siempre con Buffered*)
try (OutputStream out = new BufferedOutputStream(
    new FileOutputStream("salida.bin"))) {
    out.write(new byte[]{1, 2, 3});
}

// Lectura
try (InputStream in = new BufferedInputStream(new FileInputStream("salida.bin"))) {
    byte[] buf = new byte[8192];
    int n;
    while ((n = in.read(buf)) != -1) {          // -1 = fin de fichero
        // procesar buf[0..n-1]
    }
}

// Texto (caracteres): Reader/Writer + charset explícito
try (Reader r = new FileReader("texto.txt", StandardCharsets.UTF_8);
    BufferedReader br = new BufferedReader(r)) {
    String linea;
    while ((linea = br.readLine()) != null) { ... }
}
try (BufferedWriter w = new FileWriter("o.txt", StandardCharsets.UTF_8, true)) {
    w.write("línea"); w.newLine();
}
```

Equivalencias

byte[]/InputStream/OutputStream ↔ datos binarios · char[]/Reader/Writer ↔ texto. Tamaño de búfer típico: 8 KB. **Siempre especifica UTF-8**: FileReader sin charset usa el de la plataforma.

14.4 Serialización

```
public class Usuario implements Serializable {
    private static final long serialVersionUID = 1L;    // controla compatibilidad de versiones
    private String nombre;
    private transient String password;    // NO se serializa
}

try (ObjectOutputStream oos = new ObjectOutputStream(new FileOutputStream("u.dat"))) {
    oos.writeObject(usuario);
}
try (ObjectInputStream ois = new ObjectInputStream(new FileInputStream("u.dat"))) {
    Usuario u = (Usuario) ois.readObject();
}
```

Alternativa recomendada

La serialización nativa es lenta, frágil y con riesgos de seguridad. En proyectos reales usa **JSON** (Jackson/Gson), **YAML** o **Protocol Buffers**.

15 Concurrency y multihilo

15.1 Conceptos

- **Proceso**: programa en ejecución con su propio espacio de memoria. **Hilo (thread)**: unidad mínima de ejecución dentro de un proceso (comparte heap, tiene su propia pila y contador de instrucciones).
- **Concurrencia**: varios avanzan en el mismo intervalo. **Paralelismo**: ejecutan a la vez en núcleos distintos.
- **Estado de un hilo**: `NEW → RUNNABLE → (BLOCKED / WAITING / TIMED_WAITING) → RUNNABLE → TERMINATED`.
- **Context switch**: coste de guardar/restaurar estado → demasiados hilos = más lento.
- **Sandbox**: cada hilo comparte el heap → pueden pisarse datos (condiciones de carrera).

15.2 Crear hilos

```
// 1) Extender Thread
class MiHilo extends Thread {
    @Override public void run() { System.out.println(getName() + " corriendo"); }
}
new MiHilo().start();           // start() lanza el hilo; run() ejecutaría en el actual

// 2) Implementar Runnable (PREFERIDO: separa tarea de mecanismo de ejecución)
Runnable tarea = () -> System.out.println("tarea " + Thread.currentThread().getName());
new Thread(tarea, "worker-1").start();

// 3) Callable + FutureTask (devuelve valor y puede lanzar excepciones)
Callable<Integer> calc = () -> { return 42; };
FutureTask<Integer> ft = new FutureTask<>(calc);
new Thread(ft).start();
int r = ft.get();               // bloquea hasta que termine (get(timeout, unit) con tope)

// Utilidades
Thread.currentThread().getName();
Thread.sleep(1000);             // ms; lanza InterruptedException → hay que capturarla
hilo.join();                    // espera a que termine
hilo.setDaemon(true);           // hilo demonio: no impide que el programa acabe
```

15.3 Problemas y soluciones

Condición de carrera

Dos hilos leen-escriben un mismo recurso sin sincronizar: el resultado depende del orden de ejecución. Ejemplo clásico: `contador++` = leer → sumar → escribir (no es atómico).

```
// ❌ Inseguro
private int cuenta = 0;
public void sumar() { cuenta++; }

// ✅ Opción 1: synchronized (método o bloque)
public synchronized void sumar() { cuenta++; }
public void sumar() {
    synchronized (this) { cuenta++; } // bloque: un solo hilo a la vez
}

// ✅ Opción 2: atomic (más ligero para contadores)
private final AtomicInteger cuenta = new AtomicInteger();
cuenta.incrementAndGet();

// ✅ Opción 3: volatile (visibilidad, NO atomicidad)
private volatile boolean detenido = false;

// ✅ Opción 4: Lock explícito
private final ReentrantLock lock = new ReentrantLock();
lock.lock();
try { /* ... */ } finally { lock.unlock(); }
```

Herramienta	Garantiza
<code>synchronized</code> / <code>Lock</code>	Mutua exclusión + visibilidad
<code>volatile</code>	Visibilidad : cambios se ven de inmediato; NO hace ++ atómico
<code>Atomic*</code>	Atómicidad sin bloqueos (CAS)
<code>wait/notify</code> (dentro de <code>synchronized</code>)	Comunicación entre hilos (preferir <code>BlockingQueue</code>)
<code>CountDownLatch</code> / <code>Semaphore</code> / <code>CyclicBarrier</code>	Coordinación de puntos de encuentro

Deadlock

Dos hilos esperando bloqueados el recurso del otro. Evita: **orden fijo** al adquirir bloqueos, bloqueos cortos, `tryLock(timeout)` y no bloquear dentro de `synchronized` con llamadas a código externo.

15.4 Concurrent collections

```
ConcurrentHashMap<String, Integer> mapa = new ConcurrentHashMap<>();
mapa.putIfAbsent("k", 1);
mapa.computeIfAbsent("k", k -> contar(k)); // atómico, NO null
mapa.merge("k", 1, Integer::sum); // contador atómico idiomático

CopyOnWriteArrayList<String> lista = new CopyOnWriteArrayList<>(); // lecturas frecuentes,
// escrituras pocas

BlockingQueue<Tarea> cola = new LinkedBlockingQueue<>();
cola.put(t); // bloquea si está llena
Tarea t = cola.take(); // bloquea si está vacía
ConcurrentLinkedQueue<Tarea> noBloqueante; // offer/poll sin bloquear
```

15.5 ExecutorService y pools de hilos

```
// Usar pools: crear hilos a mano es caro y difícil de controlar
ExecutorService pool = Executors.newFixedThreadPool(4);
ExecutorService pool2 = Executors.newCachedThreadPool();
ExecutorService pool3 = Executors.newSingleThreadExecutor();
// En producción: new ThreadPoolExecutor(nucleos, max, keepAlive, unit, queue, factory)

pool.execute(() -> System.out.println("sin retorno"));
Future<Integer> f = pool.submit(() -> calcular(10));
int res = f.get(); // bloquea; get(timeout, unit) con tope

List<Future<String>> fs = new ArrayList<>();
for (Tarea t : tareas) fs.add(pool.submit(t));
for (Future<String> x : fs) resultado += x.get(); // espera todo

pool.shutdown(); // ya no acepta tareas; termina las pendientes
if (!pool.awaitTermination(30, TimeUnit.SECONDS)) pool.shutdownNow();
```

15.6 CompletableFuture (Java 8+)

```
// Encadenamiento no bloqueante
CompletableFuture<String> cf = CompletableFuture
    .supplyAsync(() -> servicio.getUsuario(id), executor)
    .thenApply(Usuario::nombre)
    .thenCompose(n -> CompletableFuture.supplyAsync(() -> buscarPerfil(n)))
    .thenApply(p -> p.toUpperCase())
    .exceptionally(ex -> { log.error("fallo", ex); return "DEFECTO"; })
    .whenComplete((v, ex) -> System.out.println("fin: " + v));

String v = cf.join(); // bloquea hasta completar (o get con timeout)

// Combinar dos llamadas en paralelo
CompletableFuture<String> a = CompletableFuture.supplyAsync(() -> "A");
CompletableFuture<String> b = CompletableFuture.supplyAsync(() -> "B");
CompletableFuture.allOf(a, b).join(); // espera a todas
CompletableFuture.anyOf(a, b).join(); // la primera en acabar
```

Método	Semántica
<code>thenApply</code>	transforma (Function) — como <code>map</code>
<code>thenCompose</code>	encadena que devuelve otra CF — como <code>flatMap</code>
<code>thenAccept</code> / <code>thenRun</code>	acción (Consumer / Runnable)
<code>handle</code> / <code>exceptionally</code> / <code>whenComplete</code>	errores y observación
<code>copy/complete/onTimeout</code> (21+)	control adicional

15.7 Virtual threads (Java 21 LTS)

```
// Hilos virtuales: miles/millones, baratos, bloqueo no consume carrier thread
Thread vt = Thread.ofVirtual().name("vt-1").start(() -> trabajo());
Thread vt2 = Thread.startVirtualThread(() -> trabajo());

// Executor de hilos virtuales (lo más habitual)
try (ExecutorService ex = Executors.newVirtualThreadPerTaskExecutor()) {
    List<Future<String>> fs = urls.stream()
        .map(u -> ex.submit(() -> descargar(u)))
        .toList();
    for (Future<String> f : fs) guardar(f.get());
}
```

Reglas de los virtual threads

- Están hechos para **I/O bloqueante** (HTTP, BD, ficheros), no para cálculo CPU-bound.
- **No los pooles**: crea uno por tarea.
- Evita `synchronized` en códigos largos con I/O → usa `ReentrantLock` o `StructuredTaskScope`.
- Siguen siendo hilos normales de la API: `Thread`, `join`, `interrupt`.

15.8 Invariantes y buenas prácticas

- Encapsula el estado compartido (nunca expongas un `ArrayList` mutable sin sincronizar).
- Haz las inmutables tu primera opción: `record`, `final`, `List.of`.
- Captura y registra las excepciones dentro de los hilos: `execute` las traga en el `UncaughtExceptionHandler`.
- Usa `Thread.currentThread().isInterrupted()` para parar bucles largos de forma limpia y restituye el flag (`Thread.currentThread().interrupt()`) si decides propagarlo.
- Mide antes de sincronizar de más: `synchronized` en un cuello de botella paraliza todo.

16 JDBC: bases de datos desde Java

```
// Dependencia (Maven): mysql-connector-j / postgresql / h2
<dependency>
  <groupId>com.mysql</groupId>
  <artifactId>mysql-connector-j</artifactId>
  <version>8.4.0</version>
</dependency>
```

16.1 Flujo básico

```
String url = "jdbc:mysql://localhost:3306/tienda?useSSL=false&serverTimezone=UTC";
String user = "root";
String pass = "secret";

try (Connection con = DriverManager.getConnection(url, user, pass)) {

    // --- SELECT con PreparedStatement (SIEMPRE; evita SQL injection) ---
    String sql = "SELECT id, nombre, precio FROM producto WHERE categoria = ? AND precio > ?";
    try (PreparedStatement ps = con.prepareStatement(sql)) {
        ps.setString(1, "electronica"); // índices empezando en 1
        ps.setDouble(2, 100.0);
        try (ResultSet rs = ps.executeQuery()) {
            while (rs.next()) { // next() devuelve false al final
                int id = rs.getInt("id");
                String nombre = rs.getString("nombre");
                double precio = rs.getDouble("precio");
            }
        }
    }

    // --- INSERT / UPDATE / DELETE ---
    try (PreparedStatement ps = con.prepareStatement(
        "INSERT INTO producto(nombre, precio) VALUES(?, ?)",
        Statement.RETURN_GENERATED_KEYS)) {
        ps.setString(1, "Portátil");
        ps.setDouble(2, 899.99);
        int filas = ps.executeUpdate(); // nº de filas afectadas
        try (ResultSet keys = ps.getGeneratedKeys()) {
            if (keys.next()) long id = keys.getLong(1);
        }
    }

    // --- Transacciones ---
    con.setAutoCommit(false);
    try {
        // ... varios executeUpdate ...
        con.commit();
    } catch (SQLException e) {
        con.rollback();
        throw e;
    } finally {
        con.setAutoCommit(true);
    }
}
```

Nunca concatenes SQL con datos

"SELECT * FROM u WHERE nombre = '" + nombre + "'" → **SQL injection** y problemas con comillas. Usa siempre PreparedStatement con ? (además comparte el plan de ejecución y es más rápido).

16.2 Buenas prácticas

- **Pool de conexiones** (HikariCP) en vez de `DriverManager` por petición.

- Cierra siempre con **try-with-resources**: `Connection`, `PreparedStatement`, `ResultSet`.
- Los recursos se cierran en orden inverso (`ResultSet` → `Statement` → `Connection`).
- Mapea resultados a objetos (records) y lanza excepciones propias en la capa de acceso a datos.
- Configura timeouts: `con.setNetworkTimeout(...)`, parámetros `connectTimeout` / `socketTimeout` de la URL.
- Usa `addBatch()` + `executeBatch()` para inserts masivos.
- Prepara el `DataSource` fuera de los bucles; reutilizar conexión es ~100× más rápido que crearla.

```
// Batch
ps = con.prepareStatement("INSERT INTO log(msg) VALUES(?)");
for (String m : mensajes) { ps.setString(1, m); ps.addBatch(); }
int[] res = ps.executeBatch();
```

17 Maven, estructura de proyecto y JUnit 5

17.1 Maven: el build por defecto

Un proyecto Maven se define por **convención de carpetas** y un `pom.xml` con el ciclo de vida: `validate` → `compile` → `test` → `package` → `verify` → `install` → `deploy`.

```
<project>
  <modelVersion>4.0.0</modelVersion>
  <groupId>com.empresa</groupId>
  <artifactId>mi-app</artifactId>
  <version>1.0.0</version>
  <packaging>jar</packaging>

  <properties>
    <maven.compiler.release>21</maven.compiler.release>
    <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
  </properties>

  <dependencies>
    <dependency>
      <groupId>org.junit.jupiter</groupId>
      <artifactId>junit-jupiter</artifactId>
      <version>5.11.3</version>
      <scope>test</scope>
    </dependency>
  </dependencies>
</project>
```

Concepto	Coordenada	Ejemplo
Quién	<code>groupId</code>	<code>com.empresa</code>
Qué	<code>artifactId</code>	<code>mi-app</code>
Qué versión	<code>version</code>	<code>1.0.0</code>
Alcance	<code>scope</code>	<code>compile</code> (defecto) · <code>test</code> · <code>provided</code> · <code>runtime</code>

```
mvn clean compile      // borra y compila
mvn test                // ejecuta los tests
mvn package             // genera el JAR (o WAR)
mvn -DskipTests package // sin tests
mvn clean install -U    // instala en ~/.m2 y actualiza snapshots
mvn dependency:tree     // árbol de dependencias (ver conflictos)
mvn archetype:generate  // plantilla de proyecto
```

- Repositorio local `~/.m2/repository`; repositorio central `repo.maven.apache.org`.
- **Gradle** es la alternativa moderna: `build.gradle` con `plugins { id 'java' }`, `./gradlew build`, `./gradlew test`. Más rápido por el daemon y el incremental build.
- Los *parent POMs* y el BOM (`dependencyManagement`) unifican versiones en proyectos multi-módulo.

17.2 JUnit 5 (Jupiter)

```
import static org.junit.jupiter.api.Assertions.*;
import org.junit.jupiter.api.*;

class CalculadoraTest {

    private Calculadora calc;

    @BeforeEach
    void preparar() { calc = new Calculadora(); }

    @AfterEach
    void limpiar() { }

    @BeforeAll static void antesDeTodo() { }
    @AfterAll static void despuesDeTodo() { }

    @Test
    void sumaCorrecta() {
        Assertions.assertEquals(5, calc.sumar(2, 3), "2+3 debe ser 5");
        Assertions.assertTrue(calc.esPositivo(1));
        Assertions.assertNull(calc.buscar("x"));
        Assertions.assertThrows(IllegalArgumentException.class,
            () -> calc.dividir(1, 0));
        Assertions.assertAll(
            () -> assertEquals(4, calc.doble(2)),
            () -> assertEquals(6, calc.doble(3)));
    }

    @ParameterizedTest
    @CsvSource({"1, 1, 2", "2, 3, 5", "-1, 1, 0"})
    void sumaParametrizada(int a, int b, int esperado) {
        assertEquals(esperado, calc.sumar(a, b));
    }

    @Disabled("pendiente de implementar")
    @Test void pendiente() { }
}
```

Anotación	Función
<code>@Test</code>	Método de prueba (sin <code>public void</code> obligatorio)
<code>@BeforeEach</code> / <code>@AfterEach</code>	Antes/después de CADA test
<code>@BeforeAll</code> / <code>@AfterAll</code>	Una vez por clase (métodos <code>static</code>)
<code>@DisplayName("texto")</code>	Nombre legible en el informe
<code>@Nested</code>	Tests anidados (contextos)
<code>@ParameterizedTest</code> + <code>@ValueSource</code> / <code>@CsvSource</code> / <code>@MethodSource</code>	Tests con datos
<code>@RepeatedTest(n)</code>	Repite el test (útil con aleatoriedad)
<code>@Timeout(2)</code>	Falla si tarda más de 2 s
<code>@Disabled</code>	Omite temporalmente

Patrón AAA

Cada test sigue **Arrange** → **Act** → **Assert**: prepara los datos, ejecuta UNA sola cosa y verifica el resultado. Un test = un comportamiento, con nombre descriptivo (`debeLanzarExcepcionSiCantidadNegativa`) y sin dependencias entre tests.

```
// Mockito: dobles de prueba
Repositorio repo = mock(Repositorio.class);
when(repo.buscar(1)).thenReturn(new Usuario("Ana"));
Servicio s = new Servicio(repo);
s.procesar(1);
verify(repo).buscar(1);           // se llamó exactamente una vez
```

18 Patrones de diseño habituales

18.1 Singleton

```
public final class Config {  
    private static final Config INSTANCIA = new Config();  
    private Config() { /* evita new desde fuera */ }  
    public static Config get() { return INSTANCIA; }  
}
```

Variante perezosa segura: `private static class Holder { static final Config I = new Config(); }` y la más robusta frente a reflexión/serialización: `enum Singleton { INSTANCIA; }`.

18.2 Factory

```
public class NotificadorFactory {  
    public static Notificador crear(Tipo t) {  
        return switch (t) {  
            case EMAIL -> new EmailNotificador();  
            case SMS    -> new SmsNotificador();  
            case PUSH   -> new PushNotificador();  
        };  
    }  
}
```

18.3 Builder (objetos con muchos parámetros)

```
public class Pedido {  
    private final String cliente;           // obligatorio  
    private final List<String> items;  
    private final String nota;  
  
    private Pedido(Builder b) {  
        this.cliente = b.cliente; this.items = b.items; this.nota = b.nota;  
    }  
  
    public static class Builder {  
        private final String cliente;  
        private List<String> items = new ArrayList<>();  
        private String nota = "";  
  
        public Builder(String cliente) { this.cliente = cliente; }  
        public Builder anadir(String i) { items.add(i); return this; }  
        public Builder nota(String n)  { this.nota = n; return this; }  
        public Pedido build()          { return new Pedido(this); }  
    }  
}  
  
Pedido p = new Pedido.Builder("Ana")  
    .anadir("Libro").nota("regalo").build();
```

18.4 Strategy

```
public interface EstrategiaDescuento { double aplicar(double total); }

public class Descuento10 implements EstrategiaDescuento {
    public double aplicar(double t) { return t * 0.9; }
}

// Con Lambda la estrategia es "un objeto con un solo método"
EstrategiaDescuento d = total -> total * 0.85;
double fin = d.aplicar(calcular());
```

18.5 Observer

```
public class PedidoObservable {
    private final List<Consumer<String>> oidos = new ArrayList<>();
    public void suscribir(Consumer<String> o) { oidos.add(o); }
    public void cambiarEstado(String e) { oidos.forEach(o -> o.accept(e)); }
}

// En la práctica: PropertyChangeSupport, @EventListener (Spring), Flux (Reactor)
```

18.6 Otros patrones que conviene conocer

Patrón	Cuándo se usa
DAO / Repository	Aislar el acceso a datos detrás de una interfaz
DTO	Transportar datos entre capas sin exponer entidades
Decorator	Añadir comportamiento sin heredar (los streams de Java lo son)
Facade	Simplificar un subsistema complejo tras una interfaz única
Template Method	Algoritmo fijo en la padre, pasos variables en las hijas
Proxy	Transparencia: caché, logging, transacciones (Spring AOP, proxies dinámicos)
Command	Encapsular una operación (deshacer, colas, auditoría)
Adapter	Traducir una interfaz a otra compatible

19 Buenas prácticas y errores frecuentes

19.1 Los 15 errores clásicos

#	Error	Corrección
1	<code>if (a == b)</code> con Strings	<code>a.equals(b)</code> o <code>Objects.equals(a, b)</code>
2	<code>NullPointerException</code>	<code>Objects.requireNonNull</code> , <code>Optional</code> , comprobar en el orden correcto
3	<code>ConcurrentModificationException</code>	<code>removeIf</code> , <code>Iterator.remove</code> o copia de la colección
4	Olvidar <code>break</code> en un switch	Usa <code>case x -></code> (Java 14+)
5	Recursos sin cerrar	try-with-resources
6	Dividir <code>int/int</code> esperando decimales	Usa <code>double</code> : <code>7.0/2</code>
7	Overflow silencioso	<code>long</code> o <code>Math.addExact</code>
8	Comparar <code>double</code> con <code>==</code>	<code>Math.abs(a-b) < EPSILON</code> ; para dinero usa <code>BigDecimal</code>
9	<code>hashCode</code> sin <code>equals</code>	Sobrescribe ambos (o usa <code>record</code> / <code>Objects.hash</code>)
10	Capturar y no registrar la excepción	<code>logger.error(msg, e)</code> o re-lanzar con la causa
11	SQL concatenando datos	<code>PreparedStatement</code> con <code>?</code>
12	Clases sin <code>toString</code>	Añádalo (o usa <code>record</code>) → logs legibles
13	Métodos gigantes	Extrae métodos con nombre de acción (~30 líneas máx.)
14	Constantes mágicas	<code>private static final</code> con nombre descriptivo
15	<code>null</code> como "sin valor"	Colección vacía, <code>Optional</code> , valor por defecto

19.2 Principios de diseño

- **Inmutabilidad primero:** `record`, campos `final`, colecciones `List.of` → código hilo-seguro por defecto.
- **Menor sorpresa:** nombres que describen intención (`buscarClientePorNif`, no `buscar2`).
- Una clase = una responsabilidad; si tiene muchos campos o nombres con "y", divídela.
- Depende de abstracciones en las fronteras: BD, red, reloj, aleatoriedad.
- Valida en los límites (constructores, endpoints) y falla rápido con mensajes claros.

19.3 Seguridad

- Nunca guardes contraseñas ni claves en el código: variables de entorno o gestor de secretos.
- Valida siempre la entrada externa y sanitiza antes de escribir en logs.
- `PreparedStatement` para SQL; escape de HTML/JSON en la capa de presentación.
- Dependencias al día: `mvn versions:display-dependency-updates` y revisión de CVEs.

19.4 Rendimiento

- **Mide antes de optimizar** (JMH, async-profiler, `jfr`).
- Evita crear objetos en bucles calientes: reutiliza, `StringBuilder`, streams con primitivos.
- Elige la estructura adecuada: `HashMap` para búsqueda, `ArrayList` para recorrido.
- Preprocesa `Pattern` fuera de bucles; usa `IntStream` para grandes volúmenes de enteros.

19.5 Logging

```
private static final Logger LOG = LoggerFactory.getLogger(MiClase.class);

LOG.info("Procesando pedido {} de {}", id, cliente); // parámetros, no concatenación
LOG.error("fallo al guardar", excepcion);           // la excepción, SIEMPRE al final
```

Niveles: `TRACE < DEBUG < INFO < WARN < ERROR`. `INFO` = eventos de negocio relevantes. Nada de `System.out.println` en producción.

19.6 Flujo de trabajo recomendado

1. Entiende el requisito y lista los casos límite (null, vacío, negativos, duplicados, límites).
2. Diseña la API de la clase *antes* de su implementación.
3. Escribe **el test primero** (TDD opcional, tests siempre).
4. Implementa lo mínimo que pase el test y refactoriza.
5. Formatea (google-java-format) y revisa con linter (Checkstyle, ErrorProne, SonarLint).
6. Documenta solo lo que el código no explica: el *por qué*, no el *qué*.

20 Chuleta rápida (cheat sheet)

20.1 Estructura mínima

```
package com.app;
import java.util.*;

public class App {
    public static void main(String[] args) { /* ... */ }
}
```

20.2 Tipos

<code>byte short int long</code>	enteros (8/16/32/64 bits)	<code>float double</code>	decimales (32/64 bits)
<code>char</code>	1 carácter Unicode (16 bits)	<code>boolean</code>	true / false
Wrappers	<code>Byte Short Integer Long Float Double Character Boolean</code> · <code>var</code> = inferencia de tipo		

20.3 Control de flujo

```
if (c) {} else {}
for (int i = 0; i < n; i++) {}
while (c) {}
break; continue; return v;

switch (x) { case a -> ...; default -> ...; }
for (T t : coleccion) {}
do {} while (c);
instanceof Tipo t
```

20.4 POO en una línea

Concepto	Sintaxis
Clase	<code>class A extends B implements C, D { }</code>
Constructor	<code>public A(T x) { this.x = x; }</code> · delegación <code>this(...)</code> · padre <code>super(...)</code>
Encapsular	<code>private</code> + getters/setters
Abstracto	<code>abstract class A { abstract void m(); }</code>
Interfaz	<code>interface I { default void m() {} }</code>
Polimorfismo	referencia padre, objeto hijo; la JVM elige el método en ejecución
Enum	<code>enum E { A, B }</code>
Record	<code>record R(String a, int b) { }</code>
Sealed	<code>sealed interface I permits X, Y</code>

20.5 Excepciones

```
try { } catch (E e) { log(e); } finally { }  
try (Recurso r = abrir()) { }           // auto-close  
throw new MiExcepcion("motivo", causa);  
throws IOException                     // en la firma
```

20.6 Colecciones esenciales

```
List<T> l = new ArrayList<>();    add get remove size sort contains  
Set<T> s = new HashSet<>();      add contains (sin duplicados)  
Map<K,V> m = new HashMap<>();    put get getOrDefault merge entrySet  
Deque<T> d = new ArrayDeque<>(); push pop peek offer poll  
Queue<T> q = new PriorityQueue<>(); offer peek poll  
List.of(...) Map.of(...)         inmutables
```

20.7 Streams en 6 líneas

```
lista.stream()  
    .filter(x -> cond)           // Predicate  
    .map(x -> transformar)       // Function  
    .sorted() .distinct() .limit(n)  
    .collect(Collectors.toList());  
// terminales: forEach count reduce min/max anyMatch findFirst toMap groupingBy joining
```

20.8 Lambdas y method references

```
(a, b) -> a + b          x -> x.length()          () -> nuevo()  
String::valueOf          System.out::println      Persona::new      lista::get
```

20.9 Concurrencia

```
new Thread(() -> tarea).start();  
ExecutorService p = Executors.newVirtualThreadPerTaskExecutor();  
p.submit(() -> calc); p.shutdown();  
CompletableFuture.supplyAsync(() -> x).thenApply(f).exceptionally(e -> y);  
synchronized (lock) { } volatile AtomicInteger ConcurrentHashMap  
Thread.ofVirtual().start(tarea);
```

20.10 Ficheros

```
Files.readString(p) · Files.write(p, txt) · Files.readAllLines(p)  
Files.lines(p) · Files.exists(p) · Files.walk(dir) · Path.of("a", "b.txt")
```


20.11 JDBC en 5 pasos

```
1. Connection con = DriverManager.getConnection(url, user, pass);
2. PreparedStatement ps = con.prepareStatement("SELECT ... WHERE id=?");
3. ps.setInt(1, id);
4. try (ResultSet rs = ps.executeQuery()) { while (rs.next()) { ... } }
5. cierre automático con try-with-resources (+ commit / rollback)
```

20.12 Comandos de consola

```
javac Archivo.java           // compilar
java Paquete.Clase           // ejecutar (usa -cp si hace falta)
java Archivo.java            // compilar y ejecutar (Java 11+)
jshell                       // REPL interactivo
jar cf app.jar -C out .      // empaquetar
mvn clean install            // build + tests
./gradlew build
```

20.13 Método de estudio recomendado

1. Lee un bloque y escribe el código **a mano** (sin copiar y pegar).
2. Modifica el ejemplo: cambia tipos, añade casos límite y lee el error que sale.
3. Escribe 3 tests con JUnit por cada clase nueva.
4. Repasa la chuleta (§20) cada semana y practica *katas*: ejercicios cortos repetibles.

Resumen en una frase

Java = **tipos estáticos + POO + JVM portátil + colecciones + Streams + concurrencia**. Dominar equals/hashCode, la jerarquía de colecciones, las excepciones y el modelo de hilos es el 80 % del día a día profesional.